

Série 2 – Solution

101.1 Programmation impérative

Question 1 – Quel est le type (au sens informatique du terme) des expressions suivantes (on suppose n entier) ?

(a) `3 % 4`

(a) Int

(b) `(10 >> 2) & 2`

(b) Int

(c) `true && (n < 5)`

(c) Boolean

(d) `"Exercise" + "3.1f"`

(d) String

(e) `if(n > 43) 4.0 else 2.0`

(e) Double

Question 2 – Soient les déclarations suivantes :

```
1 val n: Int = 10; val p: Int = 4
2 val q: Long = 2; val x: Float = 1.76f;
```

Donnez le type ainsi que la valeur des expressions suivantes :

(a) `n+q`

(a) Long, 12

(b) `n < p`

(b) Boolean, false

(c) `n % p + q`

(c) Long, 4

(d) `n+x`

(d) Float, 11.76f

(e) `n >= p`

(e) Boolean, true

(f) `n > q + 8`

(f) Boolean, false

Question 3 – Quelle est la valeur de x après l'exécution des instructions suivantes ?(a) `var x: Int = if (30 > -30) 10 % 3 else 10 % 5`(a) 1(b) `var x: Double = 0.1; x *= 45.3`(b) 4.53(c) `var x: Int = 10; x ^= 3`(c) 9(d) `var x: Int = 0xc0f0; var y: Int = 0x0a0e; x |= y`(d) 0xcafe(e) `var x: Int = 10; x /= 3`(e) 3(f) `var x: String = "Hello"; var y: String = "toto"; x+=y`(f) "Hellototo"(g) `var x: String = "Hello" + 3 + 4`(g) "Hello34"(h) `var x: String = "Hello" + (3 + 4)`(h) "Hello7"(i) `var x: Double = 3.0; x /= 3.0`(i) 1.0**Question 4 – Lesquelles de ces assignations sont valides ?**`val a: Int = 3.2`

True	False
☐	☒

`val b: Double = 4`

True	False
☒	☐

`val c: Int = (3 << 2.1).toByte`

True	False
☐	☒

`val d: Long = (121.22f).toLong`

True	False
☒	☐

`val e: Int = (24 / 21.11).toInt`

True	False
☒	☐

`val f: Char = 'c'+1;`

True	False
☐	☒

`val g: Float = (3 / 4.2);`

True	False
☐	☒

`val h: Boolean = (f > g) & 2;`

True	False

	☐	☒
<code>val i: Boolean = (e >> f) < d;</code>	<i>True</i> ☒	<i>False</i> ☐
<code>val j: Boolean = (a == c);</code>	<i>True</i> ☒	<i>False</i> ☐

Question 5 – Écrivez, lorsque cela est possible, les assignations suivantes dans leur forme courte:

(a) `x = x-1;`

(a) `x-=1`

(b) `x = x+1;`

(b) `x+=1`

(c) `x = x*4;`

(c) `x*=4`

(d) `x = x + "toto";`

(d) `x += "toto"`

(e) `x = -2;`

(e) `x = -2, pas de forme courte`

(f) `x = x / 10;`

(f) `x /= 10`

(g) `x = 10 / x;`

(g) `x = 10 / x, pas de forme courte`

Question 6 – Les parenthèses sont là surtout pour nous faciliter la lecture. Un compilateur n'a pas besoin de parenthèses. Ajoutez des parenthèses aux expressions suivantes selon la priorité des opérateurs appliquée par le compilateur.

(a) `+ a < ~ a`

(b) `-30 - 20 / 2 * 10`

(c) `-x != y + 3 * 2`

(d) `a / b * c / d`

Solution:

```
((+ a) < (~ a))
(-30) - ((20 / 2) * 10)
(-x) != (y + (3 * 2))
(((a / b) * c) / d)
```

Question 7 – Vous avez à disposition le code suivant :

1 **val** foo: Int = 0xFACE

part A l'aide des opérateurs vus au cours, faites en sorte d'afficher sur la console le contenu de la variable foo sur la console comme suit :

The value in hex is 0xface

part [**Optionnel**] Un peu plus difficile. Sans vous servir de votre ordinateur, écrivez le code pour faire en sorte d'afficher la valeur binaire comme suit. ⚠ Attention aux espaces ⚠ :

In binary it's 0b1111 1010 1100 1110

Solution:

```
1 val foo: Int = 0xFACE
2
3 println("The value in hex is 0x" + foo.toHexString)
4
5 println("In binary it's 0b"
6     + ((foo >> 12) & 0xF).toBinaryString
7     + " " + ((foo >> 8) & 0xF).toBinaryString
8     + " " + ((foo >> 4) & 0xF).toBinaryString
9     + " " + (foo & 0xF).toBinaryString)
```

Question 8 – Soient les variables suivantes :

1 **val** a: Int = 3; **val** b: Byte = 2; **val** c: Char = 10; **val** d: Double = 4.5f;

Quel est le type des expressions suivantes ?

1) a+b

1) Int

2) (d + b).toShort

2) Short

3) d * a

3) Double

4) c / b

4) Int

5) a+b+c+d

5) Double